

Ciência da Computação

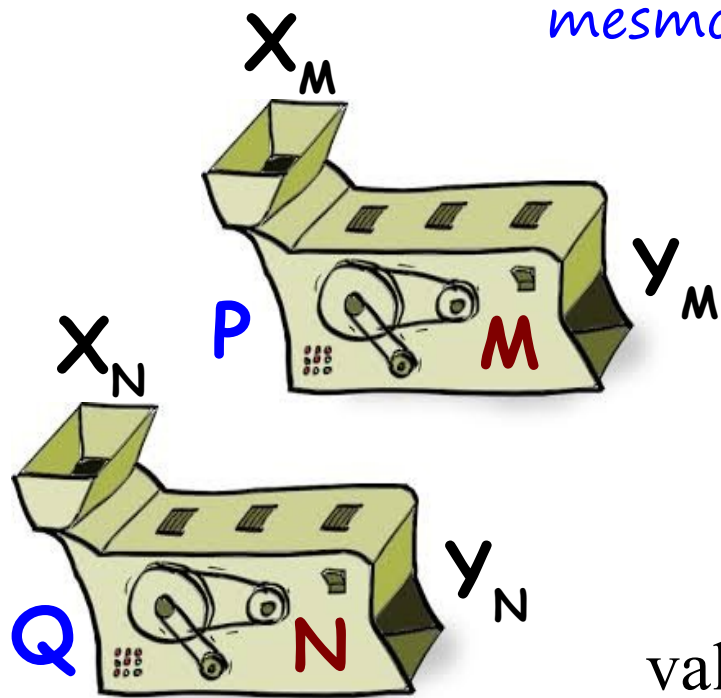
Teoria da Computação

Prof. Giovanni Ely Rocco (gerocco@ucs.br)



Equivalência de Máquinas

As máquinas podem se simular mutuamente, mesmo que a simulação use programas diferentes?



Sejam M e N duas máquinas arbitrárias,
 N simula M , se, e somente se,
para qualquer programa **P** para **M**
existe um programa **Q** para **N** ,
tais que, para os mesmos conjuntos de
valores de entrada e saída ($X_M \rightarrow X_N$ e $Y_M \rightarrow Y_N$)
as funções computadas coincidem, ou seja, $\langle P, M \rangle = \langle Q, N \rangle$;

... e **N simula fortemente M** se, e somente se,
 N simula M e M simula N .

Equivalência de Programas

*As funções computadas coincidem
para uma máquina?*

Sejam P e Q programas arbitrários
e M uma **dada** máquina

$P \equiv_M Q$ se, e somente se, $\langle P, M \rangle = \langle Q, M \rangle$.

 Relação de Equivalência de Programas
em uma Máquina

Relação de
Equivalência Forte
de Programas

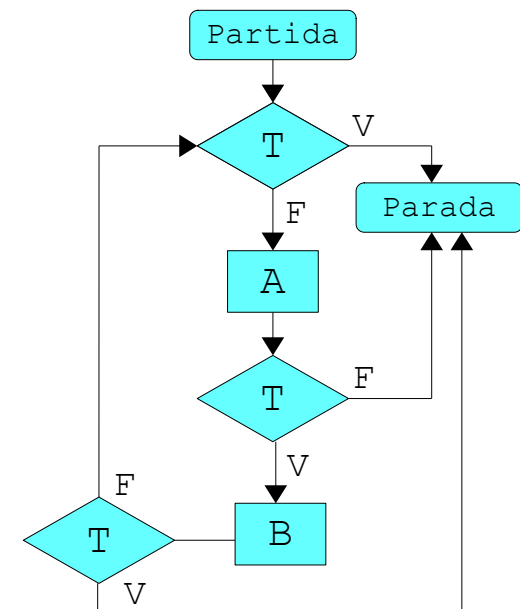
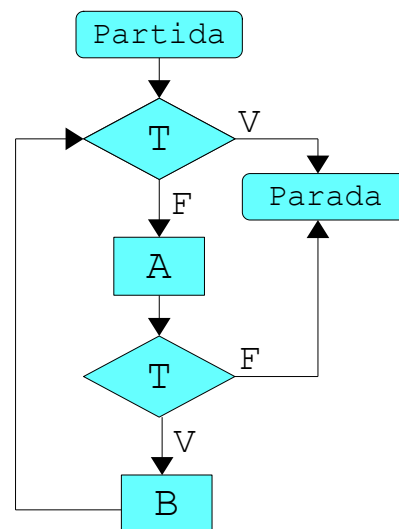
Sejam P e Q programas arbitrários
e M **qualquer** máquina

$P \equiv Q$ se, e somente se, $\langle P, M \rangle = \langle Q, M \rangle$.

Equivalência de Programas

A equivalência forte de programas:

- * permite identificar diferentes programas em uma mesma classe de equivalência; (cujas funções computadas coincidem para qualquer máquina);
- * as funções computadas por estes programas possui as mesmas operações e testes efetuados na mesma ordem;
- * fornece subsídios para analisar a complexidade estrutural de programas.



Equivalência de Programas

Equivalência Forte de Programas

* Programa Iterativo $\rightarrow$ Monolítico

Para qualquer programa iterativo P_i
existe um programa monolítico P_m , tal que $P_i \equiv P_m$

* Programa Monolítico $\rightarrow$ Recursivo

Para qualquer programa monolítico P_m
existe um programa recursivo P_r , tal que $P_m \equiv P_r$

* Programa Iterativo $\rightarrow$ Recursivo

Para qualquer programa iterativo P_i
existe um programa recursivo P_r , tal que $P_i \equiv P_r$

Equivalência de Programas

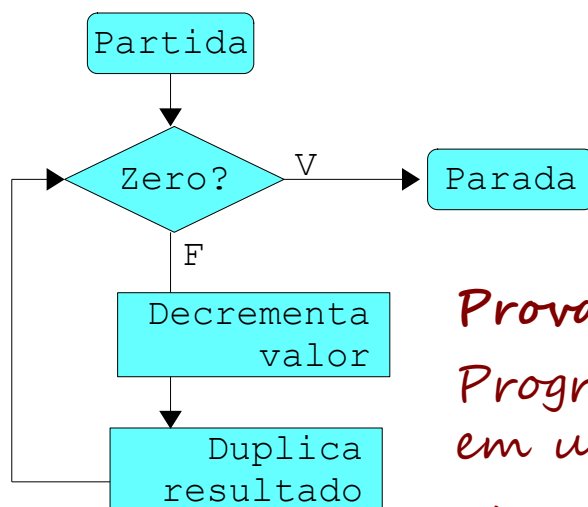
Equivalência Forte de Programas

* Programa Monolítico $\nrightarrow$ Iterativo

Dado um programa monolítico P_m qualquer
não necessariamente existe um programa iterativo P_i , tal que $P_m \equiv P_i$

* Programa Recursivo $\nrightarrow$ Monolítico

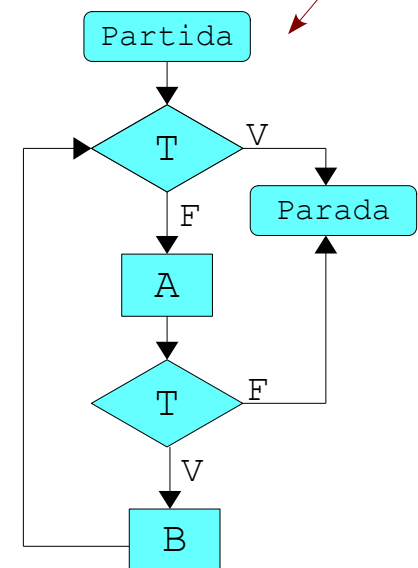
Dado um programa recursivo P_r qualquer
não necessariamente existe um programa monolítico P_m , tal que $P_r \equiv P_m$



Prova (contra-prova)?

*Programa: Duplicar um valor,
em uma máquina de um registrador.*

Monolítico? Iterativo? Recursivo?



Equivalência de Programas

Equivalência Forte de Programas

Assim, pode-se concluir que:

*para todo programa iterativo,
existe uma monolítico equivalente fortemente;*

*e para todo programa monolítico
existe um recursivo equivalente fortemente.*

Mas a inversa não é verdadeira:

*programas recursivos são
mais gerais que os monolíticos;*

*programas monolíticos são
mais gerais que os iterativos.*

Programas Recursivos

Programas Monolíticos

Programas Iterativos

Obs: As três classes de formalismos possuem o mesmo poder computacional, pois para efeitos de análise de poder computacional pode-se considerar diferentes máquinas para os diferentes programas.

Equivalência de Programas

Verificação de Equivalência Forte de Programas

* Máquina de Traços

produz um histórico (denominado 'traço') da ocorrência das operações do programa.

Dois programas são equivalentes fortemente se são equivalentes em qualquer máquina de traços.

* Programa Monolítico com Instruções Rotuladas Compostas

constituem uma forma alternativa de definir programas monolíticos.

Basicamente, uma instrução rotulada composta combina as duas formas básicas (operação e teste) em uma única forma.

A verificação de equivalência forte entre dois programas é realizada pela classe mais simples de programas, os monolíticos, pois não é conhecido um algoritmo genérico para verificação mais geral, ou seja, para os programas recursivos não se sabe se o problema é solucionável.

Equivalência de Programas

Máquina de Traços

Verificação de Equivalência Forte de Programas

$$M = (Op^*, Op^*, Op^*, id_{Op^*}, id_{Op^*}, \Pi_F, \Pi_T)$$

Op^* : Conjunto de Palavras e Operações onde $Op = \{F, G, \dots\}$
corresponde aos conjuntos de valores de memória, entrada e saída

id_{Op^*} : Função identidade em Op^*
corresponde às funções de entrada e de saída

Π_F : Conjunto de Interpretações de Operações tal que
para cada identificador de operação F de Op , $\pi_F: Op^* \rightarrow Op^*$ é tal que,
para qualquer $w \in Op^*$ resulta na concatenação de F a direita: $\pi_F(w) = wF$

Π_T : Conjunto de Interpretações de Testes tal que
para cada identificador de teste T : $\pi_T: Op^* \rightarrow \{\text{verdadeiro}, \text{falso}\}$ em Π_T

*A máquina de traços não executa operações,
o efeito de cada operação interpretada é simplesmente
acrescentar o identificador à direita do valor atual de memória:
o valor de saída consiste em um histórico das operações executadas; e
para defini-la precisa-se apenas especificar as interpretações dos testes.*

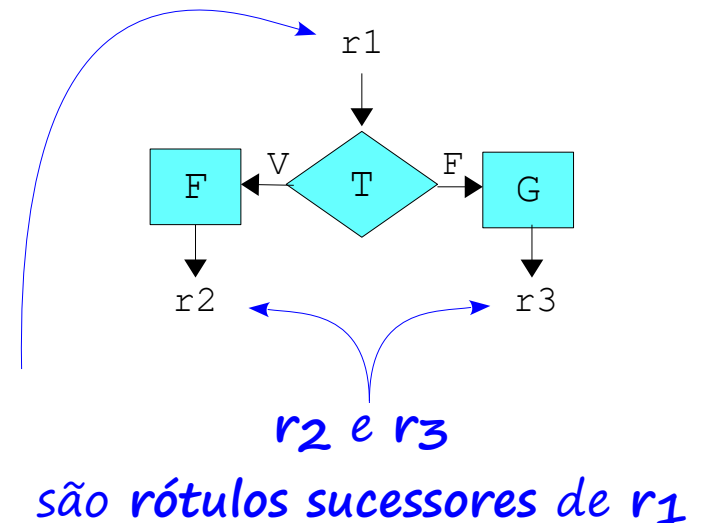
Equivalência de Programas

Instruções Rotuladas Compostas

Formato:

r_1 : se T então faça F vá_para r_2
senão faça G vá_para r_3

r_1 é rótulo antecessor
de r_2 e r_3



Em um Programa Monolítico,

o formato acima é abreviada por:

$r_1 : (F, r_2) , (G, r_3)$

Teste =
verdadeiro

Teste =
falso

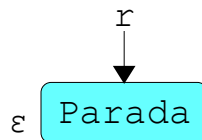
Equivalência de Programas

Instruções Rotuladas Compostas

Fluxograma → Instruções Rotuladas Compostas

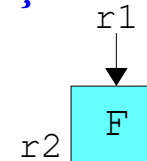
1. Rotula-se cada nó do fluxograma (partida, operação e parada);
2. o nó “partida” é o rótulo inicial;
3. segue-se o caminho do fluxograma (próxima operação executada);
4. a “parada” é identificada por palavra vazia (ϵ);

Parada



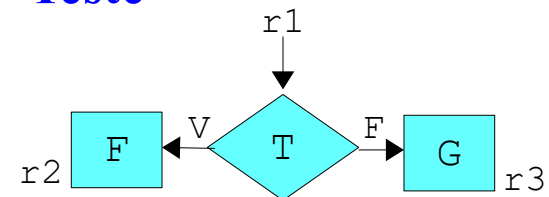
$r : (\text{parada}, \epsilon) , (\text{parada}, \epsilon)$

Operação



$r_1 : (F, r_2) , (F, r_2)$

Teste



$r_1 : (F, r_2) , (G, r_3)$

Equivalência de Programas

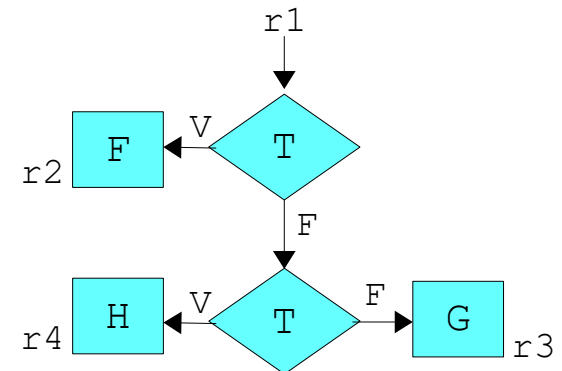
Instruções Rotuladas Compostas

Fluxograma → Instruções Rotuladas Compostas

5. nos testes, para cada caminho (V ou F)
representa-se o próximo nó (operação)
(os testes são únicos por simplificação);

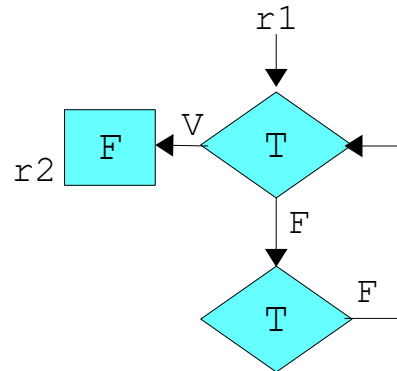
6. os ciclos devem ser representados (ω),
observando-se os
ciclos infinitos.

Testes Encadeados



$r_1 : (F, r_2) , (G, r_3)$

Testes Encadeados em Ciclo Infinito



$r_1 : (F, r_2) , (\text{ciclo}, \omega)$
 $\omega : (\text{ciclo}, \omega) , (\text{ciclo}, \omega)$

Verificação de Equivalência de Programas

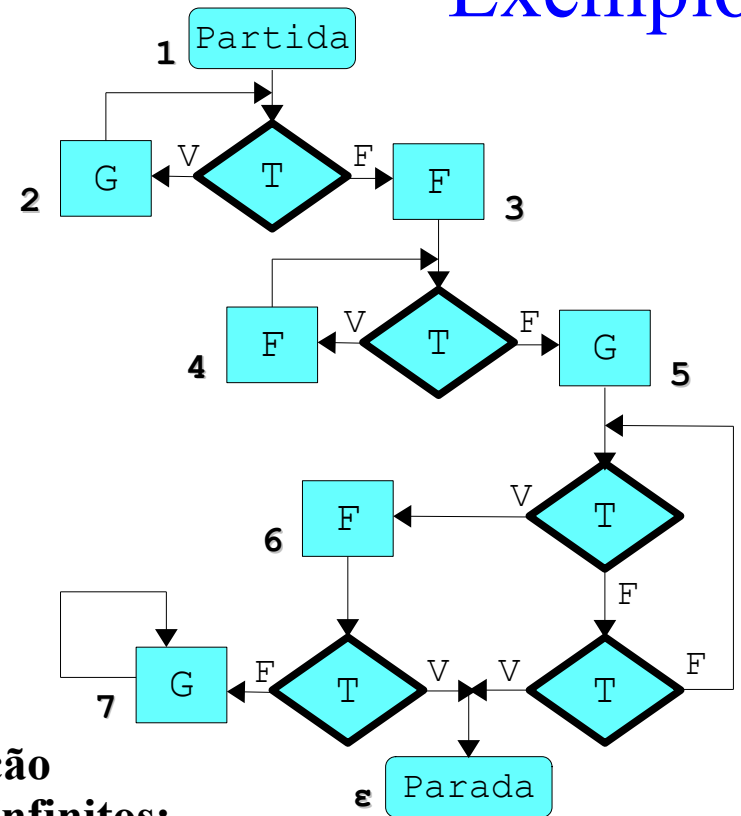
Exemplo

Instruções Rotuladas Compostas

1: (G, 2) , (F, 3)
2: (G, 2) , (F, 3)
3: (F, 4) , (G, 5)
4: (F, 4) , (G, 5)
5: (F, 6) , (ciclo, ω)
6: (parada, ϵ) , (G, 7)
7: (G, 7) , (G, 7)

Instruções Rotuladas Compostas Simplificada

1: (G, 2) , (F, 3)
2: (G, 2) , (F, 3)
3: (F, 4) , (G, 5)
4: (F, 4) , (G, 5)
5: (F, 6) , (ciclo, ω)
6: (parada, ϵ) , (ciclo, ω)
~~7: (G, 7) , (G, 7)~~
 ω : (ciclo, ω) , (ciclo, ω)



Identificação de Ciclos Infinitos:

$A_0 = \{ \epsilon \}$

$A_1 = \{ 6, \epsilon \}$

$A_2 = \{ 5, 6, \epsilon \}$

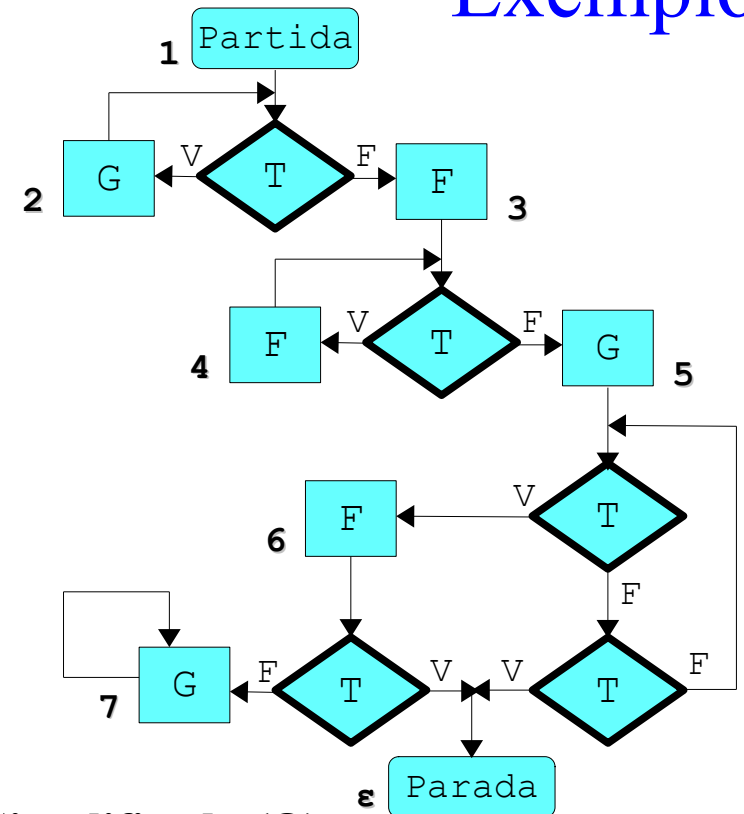
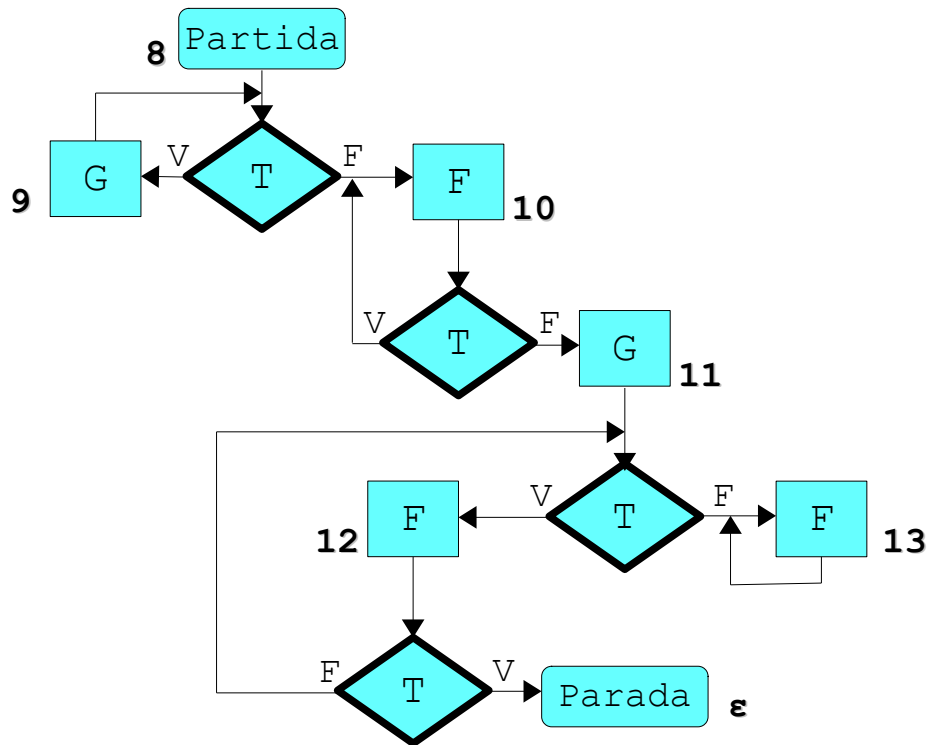
$A_3 = \{ 3, 4, 5, 6, \epsilon \}$

$A_4 = \{ 1, 2, 3, 4, 5, 6, \epsilon \}$

$A_5 = \{ 1, 2, 3, 4, 5, 6, \epsilon \}$

Verificação de Equivalência de Programas

Exemplo



IR Composta Simplificada (R)

8: (G, 9) , (F, 10)
 9: (G, 9) , (F, 10)
 10: (F, 10) , (G, 11)
 11: (F, 12) , (ciclo, ω)
 12: (parada, ε) , (ciclo, ω)
 ω: (ciclo, ω) , (ciclo, ω)

IR Composta Simplificada (Q)

1: (G, 2) , (F, 3)
 2: (G, 2) , (F, 3)
 3: (F, 4) , (G, 5)
 4: (F, 4) , (G, 5)
 5: (F, 6) , (ciclo, ω)
 6: (parada, ε) , (ciclo, ω)
 ω: (ciclo, ω) , (ciclo, ω)

Verificação de Equivalência de Programas

Exemplo

União Disjunta I_Q e I_R

1: (G, 2) , (F, 3)
2: (G, 2) , (F, 3)
3: (F, 4) , (G, 5)
4: (F, 4) , (G, 5)
5: (F, 6) , (ciclo, ω)
6: (parada, ϵ) , (ciclo, ω)
8: (G, 9) , (F, 10)
9: (G, 9) , (F, 10)
10: (F, 10) , (G, 11)
11: (F, 12) , (ciclo, ω)
12: (parada, ϵ) , (ciclo, ω)
 ω : (ciclo, ω) , (ciclo, ω)

Para verificar se $Q \equiv R$
basta verificar se $(I, 1) \equiv (I, 8)$:

$B_0 = \{ (1, 8) \}$

$B_1 = \{ (2, 9), (3, 10) \}$

$B_2 = \{ (4, 10), (5, 11) \}$

$B_3 = \{ (6, 12), (\omega, \omega) \}$

$B_4 = \{ (\epsilon, \epsilon) \}$

$B_5 = \emptyset$

Logo $(I, 1) \equiv (I, 8)$ e, portanto, $Q \equiv R$

Rótulos Consistentes e Rótulos Equivalentes Fortemente

$r: (F_1, r_1) , (F_2, r_2)$

$s: (G_1, s_1) , (G_2, s_2)$

São consistentes se,
e somente se,

$F_1 = G_1$ e $F_2 = G_2$

São equivalentes fortemente se,
e somente se,

$r = s = \epsilon$

ou $r \neq \epsilon$ e $s \neq \epsilon$

e r e s são consistentes

Exercícios

Considerando os seguintes programas...

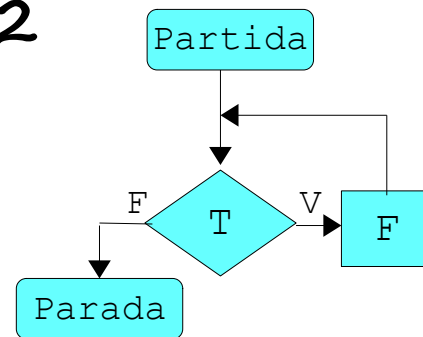
Programa 1

enquanto T
 faça (F)

Programa 3

P4 é R onde
 R def se T então (F;R) senão √

Programa 2



São equivalentes?

Exercícios

Considerando os seguintes programas...

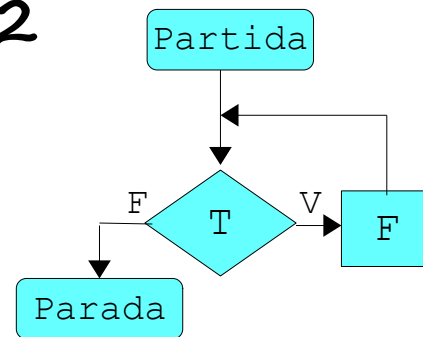
Programa 1

enquanto T
 faça (F)

Programa 3

P4 é R onde
 R def se T então (F;R) senão √

Programa 2



São equivalentes?

Instruções Rotuladas Compostas

1 : (F, 2) , (parada, ε)

2 : (F, 2) , (parada, ε)

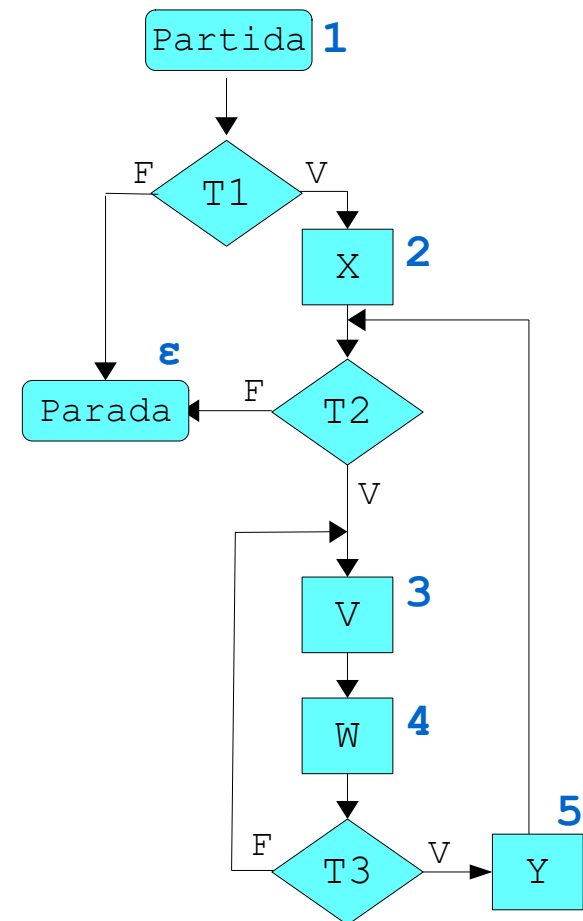
Exercícios

Considerando o seguinte programa...

```
se T1
  então (X; enquanto T2
        faça (faça (V;W)
              até T3; Y))
  senão (✓)
```

Programa monolítico?

Instruções Rotuladas Compostas?



Exercícios

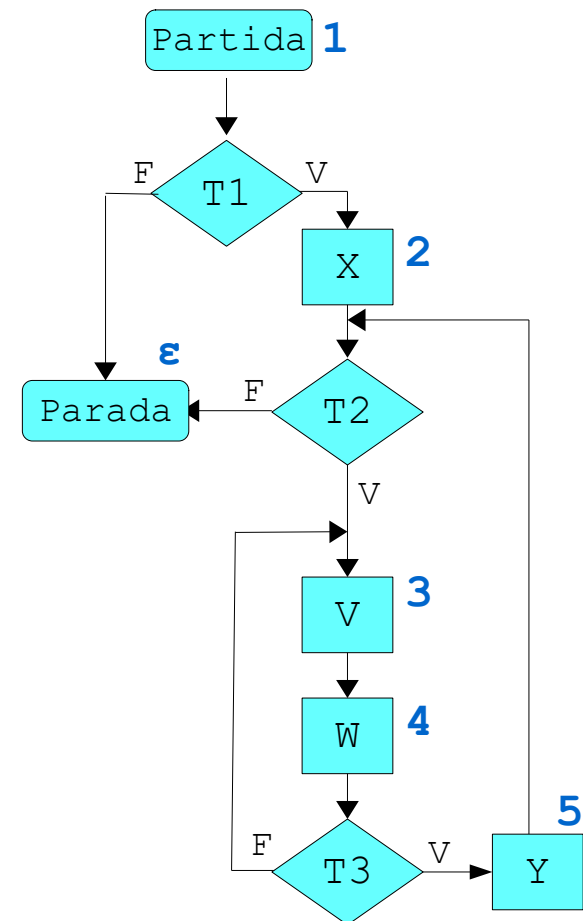
Considerando o seguinte programa...

```
se T1
  então (X; enquanto T2
        faça (faça (V;W)
              até T3; Y))
  senão (✓)
```

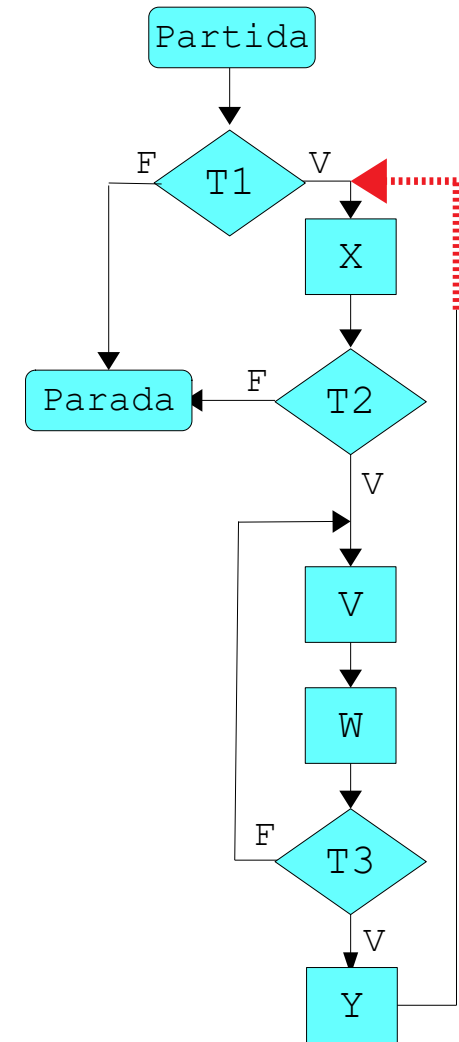
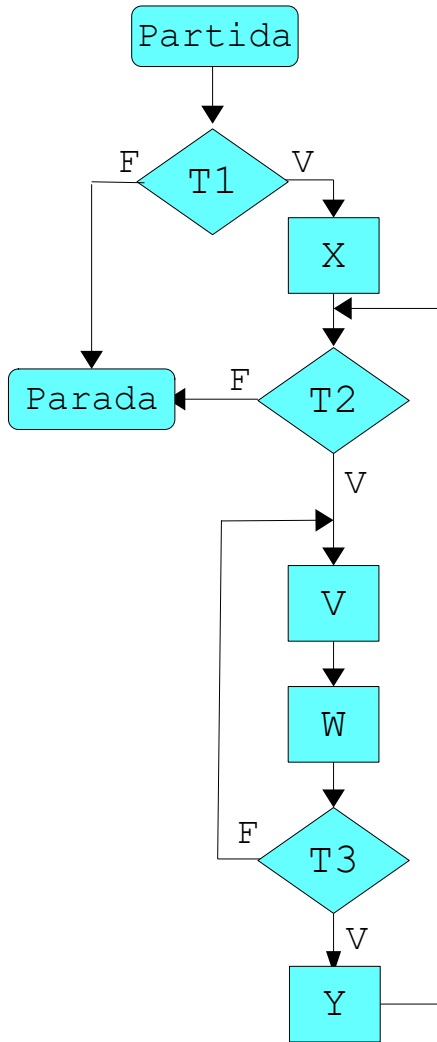
Programa monolítico?

Instruções Rotuladas Compostas?

1: (X, 2), (parada, ϵ)
2: (V, 3), (parada, ϵ)
3: (W, 4), (W, 4)
4: (Y, 5), (V, 3)
5: (V, 3), (parada, ϵ)



Exercícios



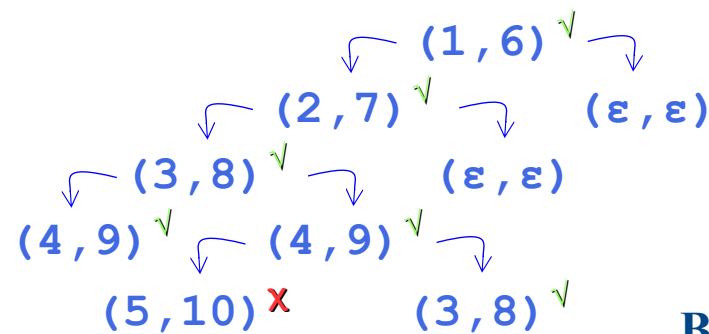
Demonstrar que esses programas não são equivalentes.

Exercícios

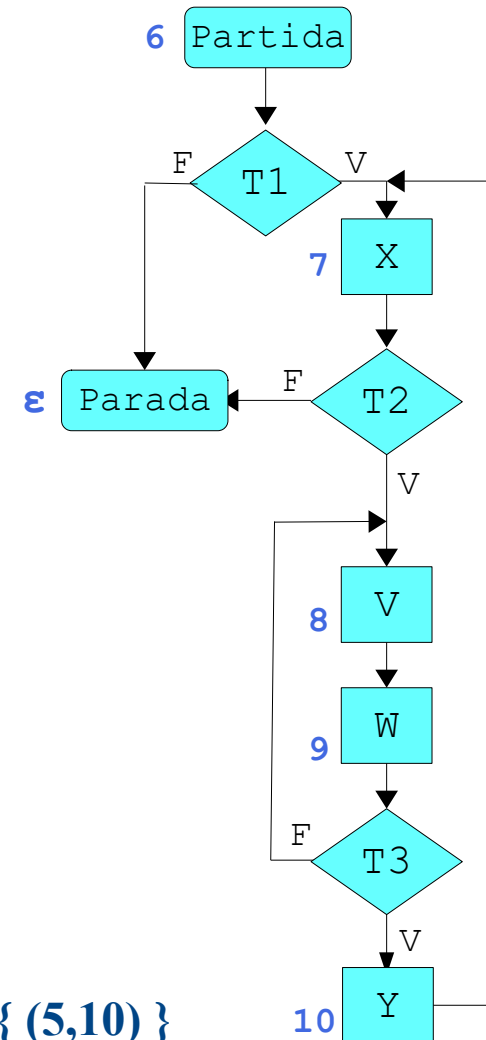
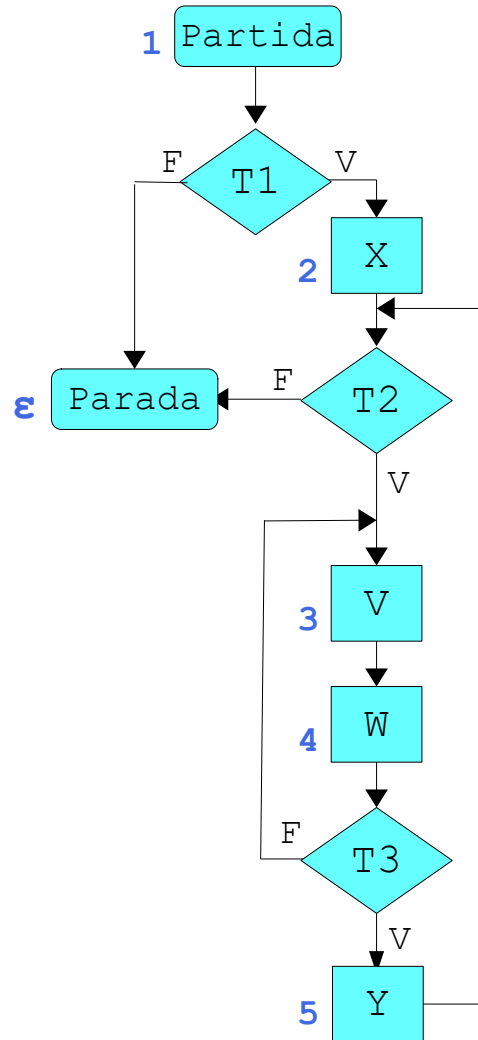
Instruções rotuladas compostas:

1: (X, 2) , (parada, ϵ)
 2: (V, 3) , (parada, ϵ)
 3: (W, 4) , (W, 4)
 4: (Y, 5) , (V, 3)
 5: (V, 3) , (parada, ϵ)
 6: (X, 7) , (parada, ϵ)
 7: (V, 8) , (parada, ϵ)
 8: (W, 9) , (W, 9)
 9: (Y, 10) , (V, 8)
 10: (X, 7) , (X, 7)

Rótulos Consistentes



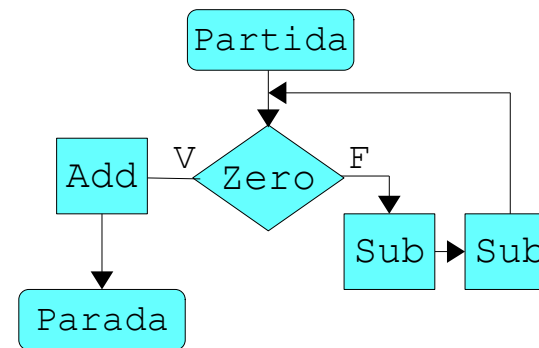
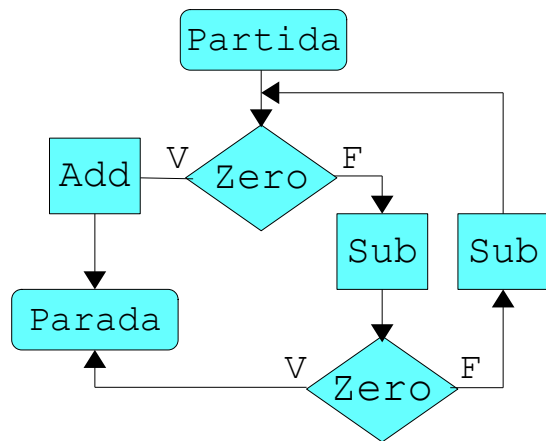
$B = \{ (5,10) \}$



Conclui-se que os programas não são equivalentes.

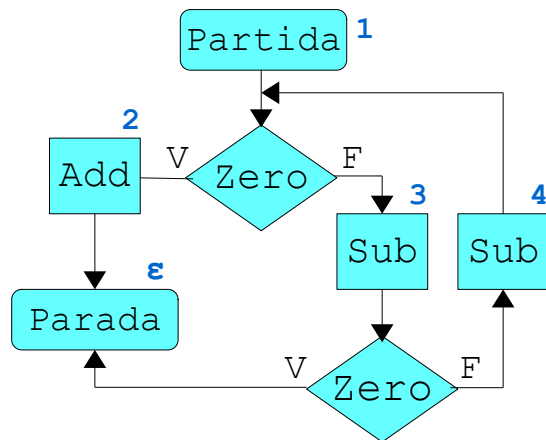
Exemplo

Os programas são equivalentes?

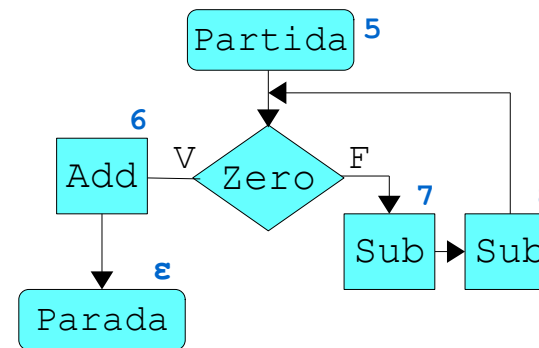


Exemplo

Os programas são equivalentes?



Este programa retorna zero se ímpar e um se par.



Este programa não funciona para o mesmo propósito, pois apresenta problema de estruturação lógica (com estruturação iterativa não é possível nessa máquina).

Instruções rotuladas compostas:

1: (Add, 2) , (Sub, 3)
2: (parada, ε) , (parada, ε)
3: (parada, ε) , (Sub, 4)
4: (Add, 2) , (Sub, 3)

Rótulos Consistentes:

5: (Add, 6) , (Sub, 7) (1, 5) ✓
6: (parada, ε) , (parada, ε) (2, 6) ✓
7: (Sub, 8) , (Sub, 8) (3, 7) ✗
8: (Add, 6) , (Sub, 7)

Como $(1,1) \neq (1,5)$, os programas não são equivalentes!